Task 1: String Formatting in Python

1. Introduction

String formatting is the process of creating strings by combining fixed text with variable data. Python provides several formatting techniques, including `%` formatting, `str.format()`, f-strings, and `string.Template`. Modern Python applications generally prefer f-strings because they are readable, concise, and powerful.

---

2. `%` Formatting

The `%` operator is the oldest major string-formatting technique in Python. It is similar to the `printf()` style used in C.

```
name = "Ali"
age = 20

print("My name is %s and I am %d years old." % (name, age))
```

Common specifiers include:

- `%s` → string
- `%d` → integer
- `%f` → floating-point number
- `%x` → hexadecimal
- `%e` → scientific notation
- `%` → percentage symbol

Example:

```
price = 25.6789
print("Price: %.2f" % price)
```

Output:

```
Price: 25.68
```

Advantages

- Simple and familiar to programmers coming from C.
- Compatible with old Python code.
- Useful for maintaining legacy projects.

Disadvantages

- Less readable for complex formatting.
- Can become confusing with many variables.
- Has fewer features than modern formatting methods.

---

3. `str.format()`

The `format()` method was introduced to provide a more flexible formatting system.

```
name = "Sara"
age = 21

print("My name is {} and I am {} years old.".format(name, age))
```

Named arguments can also be used:

```
print("My name is {name} and I am {age}.".format(name="Sara", age=21))
```

It supports advanced formatting such as alignment, padding, precision, percentages, and different number bases. Python's format specification mini-language supports options for width, alignment, signs, precision, grouping, and numeric presentation.

---

4. F-Strings

F-strings were introduced in Python 3.6 and are the preferred general-purpose formatting technique in modern Python.

```
name = "Sara"
age = 21

print(f"My name is {name} and I am {age} years old.")
```

Expressions can be placed directly inside `{}`:

```
a = 10
b = 20

print(f"Sum = {a + b}")
```

Formatting is also easy:

```
price = 1234.5678
print(f"Price: {price:.2f}")
```

F-strings are usually the most readable option because the variables appear directly beside the text they represent.

---

5. Template Strings

Python's `string.Template` provides a simpler `$`-based substitution system.

```
from string import Template

template = Template("Hello $name, welcome to $city.")

print(template.substitute(name="Sara", city="Cairo"))
```

`safe_substitute()` can be used when missing values should remain unchanged instead of raising an exception.

```
template = Template("Hello $name, your age is $age.")
print(template.safe_substitute(name="Sara"))
```

Template strings are especially useful when the template may be edited by non-programmers or when a simpler substitution syntax is desirable.

Important modern Python note

Python 3.14 also introduced **template string literals**, written with a `t` prefix such as:

```
name = "Sara"
message = t"Hello {name}!"
```

Unlike an f-string, a template string literal produces a `Template` object containing static and interpolated parts, which can then be processed by specialized code.

---

6. Important Format Specifiers

Python's modern formatting system supports many useful specifications.

Floating-point precision

```
number = 3.14159265
print(f"{number:.2f}")
```

Output:

```
3.14
```

Alignment

```
name = "Python"

print(f"{name:<10}")   # Left
print(f"{name:>10}")   # Right
print(f"{name:^10}")   # Center
```

Padding

```
number = 42

print(f"{number:05d}")
```

Output:

```
00042
```

Thousands separator

```
number = 1234567
print(f"{number:,}")
```

Output:

```
1,234,567
```

Percentage

```
score = 0.875
print(f"{score:.2%}")
```

Output:

```
87.50%
```

Hexadecimal

```
number = 255
print(f"{number:x}")
```

Output:

```
ff
```

Binary

```
number = 10
print(f"{number:b}")
```

Output:

```
1010
```

Scientific notation

```
number = 123456.789
print(f"{number:.2e}")
```

Dates

```
from datetime import datetime

today = datetime.now()
print(f"{today:%Y-%m-%d}")
```

The formatting mini-language also supports decimal, octal, binary, hexadecimal, scientific notation, percentages, alignment, grouping, and other options.

---

7. Custom Object Formatting

Python objects can customize their formatting behavior by implementing `__format__()`.

```
class Student:
    def __init__(self, name):
        self.name = name

    def __format__(self, spec):
        return f"Student: {self.name}"

student = Student("Sara")
print(f"{student}")
```

This allows classes to define how their objects should appear when used with `format()` or f-strings.

---

8. Comparison of Formatting Techniques

Method	Readability	Flexibility	Performance	Compatibility
`%`	Low/Medium	Medium	Good	Very high

Method	Readability	Flexibility	Performance	Compatibility
`str.format()`	Good	High	Good	High
f-strings	Excellent	Very high	Excellent	Python 3.6+
`Template`	Good	Low/Medium	Usually lower	High
t-strings	Excellent for structured processing	High	Newer feature	Python 3.14+

In general, f-strings are preferred for normal application code. `str.format()` remains useful when the format string must be separated from the values or when compatibility with older Python versions is required. `Template` is useful for simple user-editable templates.

Performance depends on Python version, expression complexity, and how the benchmark is designed. In typical modern CPython usage, f-strings are among the fastest approaches because their expressions are integrated into Python's formatting machinery. Exact timings should be measured with `timeit` on the target Python version rather than treated as universal numbers.

Example benchmark:

```
import timeit

print(timeit.timeit(
    '"Hello {}".format("Python")',
    number=1_000_000
))

print(timeit.timeit(
    'f"Hello {"Python"}"',
    number=1_000_000
))

print(timeit.timeit(
    '"Hello %s" % "Python"',
    number=1_000_000
))
```

---

9. Common Mistakes

1. Forgetting the correct format specifier.
2. Using `%` formatting in new code when f-strings would be clearer.
3. Forgetting that `{}` has special meaning in `format()`.
4. Using the wrong precision, such as confusing decimal places with significant digits.
5. Performing unnecessary calculations inside a format expression.
6. Using string formatting as a replacement for proper data validation.

Task 2: Procedural vs Functional vs Object-Oriented Programming

1. Introduction

Programming paradigms are different approaches to organizing and solving software problems. Three important paradigms are **Procedural Programming, Functional Programming, and Object-Oriented Programming (OOP)**.

Python supports all three, while languages such as Java emphasize OOP, Haskell emphasizes functional programming, and C is strongly procedural.

---

2. Procedural Programming

Procedural programming organizes a program as a sequence of procedures or functions that operate on data.

Main characteristics

- Functions and procedures
- Step-by-step execution
- Mutable program state
- Shared data may be used
- Focus on algorithms and actions

Example:

```
def calculate_total(numbers):
    total = 0

    for number in numbers:
        total += number

    return total

numbers = [10, 20, 30]
print(calculate_total(numbers))
```

Advantages

- Simple and easy to understand.

- Effective for small and medium programs.
- Usually has low conceptual overhead.
- Works well for algorithmic tasks.

Disadvantages

- Large programs can become difficult to maintain.
- Shared mutable state can create bugs.
- Data and operations are not always strongly separated.

---

3. Functional Programming

Functional programming treats computation as the evaluation of functions. It emphasizes **pure functions, immutability, function composition, and minimizing side effects**.

A pure function produces the same output for the same input and does not modify external state.

```
numbers = [10, 20, 30]

def square(x):
    return x * x

result = list(map(square, numbers))
print(result)
```

Another example:

```
numbers = [1, 2, 3, 4, 5]

even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
```

Advantages

- Easier testing because pure functions are predictable.
- Reduced problems caused by shared state.
- Good for parallel and concurrent processing.
- Encourages reusable functions.

Disadvantages

- Can be harder for beginners.
- Excessive use of functional constructs can reduce readability.
- Some problems are naturally easier to model using objects or state.

---

4. Object-Oriented Programming

OOP organizes software around **objects**, which combine data and behavior.

Important OOP concepts include:

- Classes
- Objects
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

Example:

```
class Student:
    def __init__(self, name, grades):
        self.name = name
        self.grades = grades

    def average(self):
        return sum(self.grades) / len(self.grades)

student = Student("Sara", [90, 85, 95])

print(student.average())
```

Encapsulation

Encapsulation keeps related data and behavior together and controls how data is accessed.

Inheritance

Inheritance allows a class to reuse or extend another class.

```
class Animal:
    def speak(self):
        print("Animal sound")

class Dog(Animal):
    def speak(self):
        print("Woof")
```

Polymorphism

Different objects can provide the same operation in different ways.

```
animals = [Dog(), Animal()]
```

```
for animal in animals:
    animal.speak()
```

---

5. Same Problem Using Three Paradigms

Suppose we need to calculate the total price of products.

Procedural

```
prices = [10, 20, 30]
total = 0

for price in prices:
    total += price

print(total)
```

Functional

```
prices = [10, 20, 30]

total = sum(map(lambda x: x, prices))
print(total)
```

A more meaningful functional example could be:

```
prices = [10, 20, 30]

discounted = list(map(lambda x: x * 0.9, prices))
total = sum(discounted)

print(total)
```

Object-Oriented

```
class Cart:
    def __init__(self, prices):
        self.prices = prices

    def total(self):
        return sum(self.prices)

cart = Cart([10, 20, 30])
print(cart.total())
```

The three solutions produce the same type of result but organize the problem differently.

---

6. Paradigms Across Programming Languages

Language	Procedural	Functional	OOP
Python	Yes	Yes	Yes
C	Strong	Limited	No traditional OOP
C++	Yes	Yes	Strong
Java	Limited	Yes	Strong
C#	Yes	Strong support	Strong
JavaScript	Yes	Strong	Prototype-based OOP
Go	Yes	Some functional features	No traditional classes
Rust	Yes	Strong functional features	Trait-based rather than traditional class OOP
Haskell	Limited	Very strong	Not traditional OOP

Modern languages are often multi-paradigm rather than belonging to only one category.

---

7. Comparison

Feature	Procedural	Functional	OOP
Main focus	Procedures	Functions	Objects
State	Often mutable	Prefer immutable	Encapsulated
Side effects	Common	Minimized	Controlled
Reusability	Functions	Pure functions	Classes/components
Scalability	Medium	High	High
Learning difficulty	Low	Medium/High	Medium
Best for	Algorithms, scripts	Data processing	Large systems

Conclusion

There is no universally best paradigm. Procedural programming is useful for straightforward algorithms and scripts. Functional programming is excellent when predictable transformations and limited side effects are important. OOP is useful when a system contains many interacting entities with data and behavior.

Modern software frequently combines paradigms. Python, for example, can use procedural code, functional techniques, and OOP in the same project.

Task 3: Clean Code

1. Introduction

Clean Code is a software development philosophy focused on writing code that is easy to read, understand, test, modify, and maintain.

The goal is not simply to make code work. The goal is to make the code understandable to the developers who will maintain it later.

---

2. Meaningful Naming

Names should clearly describe their purpose.

Bad

```
x = 10
y = 20
z = x + y
```

Better

```
price = 10
tax = 20
total_price = price + tax
```

Meaningful names reduce the need for explanations.

---

3. Small Functions

Functions should generally have one clear responsibility.

Bad

```
def process_student(student):
    print(student.name)
    calculate_grades(student)
    save_to_database(student)
    send_email(student)
```

This function performs several unrelated responsibilities.

Better

```
def display_student(student):
    print(student.name)

def save_student(student):
    save_to_database(student)

def notify_student(student):
    send_email(student)
```

Small functions are easier to test and modify.

---

4. Avoid Duplication

Repeated code increases maintenance costs.

Bad

```
total1 = price1 + price1 * 0.14
total2 = price2 + price2 * 0.14
```

Better

```
def calculate_total(price):
    return price * 1.14

total1 = calculate_total(price1)
total2 = calculate_total(price2)
```

This follows the DRY principle: **Don't Repeat Yourself**.

---

5. Comments

Comments should explain important reasoning rather than obvious code.

Bad

```
# Add 1 to count
count += 1
```

The comment provides no useful information.

Better

```
# Start counting from 1 because the report uses human-readable numbering.
count += 1
```

Good code should be understandable without excessive comments.

---

6. Code Organization and Formatting

Clean code should have:

- Consistent indentation.
- Logical file organization.
- Short and focused functions.
- Consistent naming conventions.
- Reasonable line length.
- Related code grouped together.

Python projects should generally follow PEP 8 style guidelines.

---

7. Error Handling

Errors should be handled clearly and at the appropriate level.

Bad

```
try:
    result = calculate()
except:
    pass
```

This hides errors and makes debugging difficult.

Better

```
try:
    result = calculate()
except ValueError as error:
    print(f"Invalid value: {error}")
```

Specific exceptions should be handled instead of catching every possible exception.

---

8. Testing

Clean code should be testable.

Example:

```
def add(a, b):
    return a + b
```

Test:

```
assert add(2, 3) == 5
assert add(0, 5) == 5
```

Automated tests make refactoring safer and help prevent regressions.

---

9. Refactoring

Refactoring means improving the internal structure of code without changing its external behavior.

Examples include:

- Renaming variables.
- Splitting large functions.
- Removing duplication.
- Simplifying complex conditions.
- Improving class structure.

Conclusion

Clean Code focuses on readability, simplicity, maintainability, and reliability. Good code should communicate its purpose clearly and make future changes easier.

Task 4: Uncle Bob (Robert C. Martin)

1. Introduction

Robert C. Martin, commonly known as **Uncle Bob**, is a well-known software engineer, author, and speaker. He has had a major influence on software craftsmanship, Agile development, Clean Code, and software architecture.

His philosophy emphasizes that professional developers should take responsibility for the quality of their software rather than focusing only on whether the program currently works.

---

2. Important Books

Clean Code

Clean Code: A Handbook of Agile Software Craftsmanship focuses on writing readable, maintainable, and understandable source code.

Major ideas include:

- Meaningful names.
- Small functions.
- Avoiding duplication.
- Clear error handling.
- Simple designs.
- Continuous refactoring.
- Testable code.

Clean Architecture

Clean Architecture explains how software systems should be structured so that important business rules remain independent of frameworks, databases, and external technologies.

A major idea is the **Dependency Rule**:

Dependencies should point toward higher-level business rules.

The architecture commonly contains layers such as:

1. Entities
2. Use Cases
3. Interface Adapters
4. Frameworks and Drivers

The purpose is to make the core of the system independent of external details.

---

3. The Clean Coder

The Clean Coder focuses on professionalism and developer behavior.

Important lessons include:

- Take responsibility for your work.
- Communicate clearly.
- Manage time effectively.
- Avoid making unrealistic promises.
- Practice continuously.
- Write tests.
- Refactor regularly.
- Maintain professional standards.

Programming is presented not only as a technical activity but also as a professional discipline.

---

4. Agile Software Development: Principles, Patterns, and Practices

This book connects Agile development with practical software engineering principles.

It discusses:

- Agile principles.
- Design patterns.
- Refactoring.
- Unit testing.
- Software design.
- Object-oriented principles.
- Iterative development.

The main idea is that Agile development should be supported by strong engineering practices rather than simply following a project-management process.

---

5. SOLID Principles

SOLID is a group of five principles for designing maintainable object-oriented software.

S: Single Responsibility Principle

A class should have one primary responsibility.

Bad

```
class Employee:
    def calculate_salary(self):
        pass

    def save_to_database(self):
        pass

    def send_email(self):
        pass
```

The class has multiple responsibilities.

Better

```
class SalaryCalculator:
    def calculate(self):
        pass

class EmployeeRepository:
    def save(self):
        pass

class EmailService:
    def send(self):
        pass
```

O: Open/Closed Principle

Software entities should be open for extension but closed for modification.

Instead of constantly modifying existing code, new behavior should ideally be added through new components or implementations.

L: Liskov Substitution Principle

Objects of a subclass should be usable wherever objects of the parent class are expected without breaking the program.

In simple terms, a child class should properly behave like its parent type.

I: Interface Segregation Principle

Clients should not be forced to depend on methods they do not use.

Large interfaces should often be divided into smaller, focused interfaces.

D: Dependency Inversion Principle

High-level modules should not depend directly on low-level implementation details. Both should depend on abstractions.

For example:

```
class NotificationService:
    def __init__(self, sender):
        self.sender = sender
```

The service depends on a sender abstraction rather than directly depending on a specific email provider.

---

6. Clean Architecture Example

Consider an online shopping application.

A poor architecture might directly connect business logic to a specific database:

```
Business Logic → MySQL
```

If the database changes, the business logic must also change.

A cleaner architecture is:

```
User Interface
      ↓
Controllers
      ↓
Use Cases
      ↓
Business Entities
      ↑
Database / External Services
```

The business rules remain independent from external technologies.

This makes the system easier to test, maintain, and modify.

---

7. Key Lessons from Uncle Bob

The most important lessons from Robert C. Martin's work include:

1. **Code should be readable.**
2. **Functions and classes should have clear responsibilities.**
3. **Avoid unnecessary complexity.**
4. **Do not duplicate code unnecessarily.**
5. **Write automated tests.**
6. **Refactor continuously.**
7. **Use good architecture to isolate business rules from external details.**
8. **Depend on abstractions when appropriate.**
9. **Develop software professionally and take responsibility for its quality.**
10. **Good software is designed for change, not just for today's requirements.**
